

Practical 1

Aim:

To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Bubble sort

Algorithm:

① Input:- Array Elements
output:- sorted Elements

step ①:- start

step ②:- repeat for $i=0$ to $n-2$

step ③:- repeat for $j=0$ to $n-2-i$

step ④:- compare $A[j]$ and $A[j+1]$

step ⑤:- if $A[j] > A[j+1]$

step ⑥:- swap $A[j]$ and $A[j+1]$

step ⑦:- End if

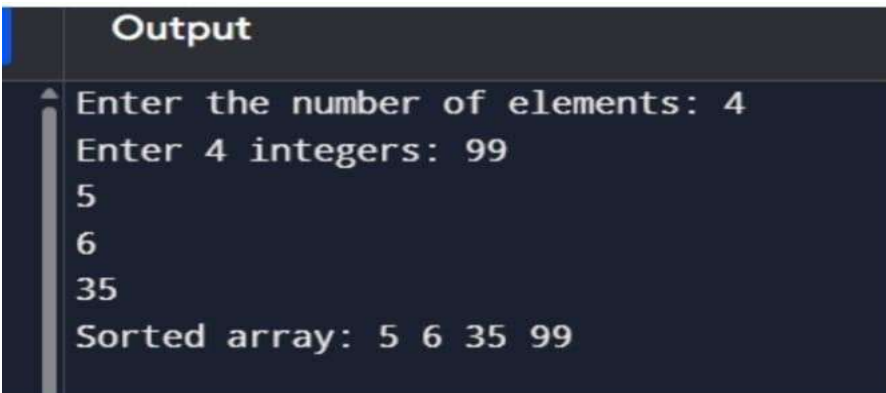
step ⑧:- End for

step ⑨:- stop

Code for bubble sort:

```
#include <iostream> #include
<vector> using namespace std; void
bubbleSort(vector<int>& arr) { int
n = arr.size(); for (int i = 0; i < n -
1; ++i) { for (int j = 0; j < n - i -
1; ++j) { if (arr[j] > arr[j
+ 1]) { int temp = arr[j]; arr[j]
= arr[j + 1]; arr[j +
1] = temp;
}
}
} } int main() { int size; cout <<
"Enter the
number of elements: "; cin >> size;
vector<int> arr(size); cout << "Enter " <<
size << " integers: "; for (int i = 0; i < size;
++i) { cin >> arr[i];
}
bubbleSort(arr); cout <<
"Sorted array: "; for (int i = 0;
i < size; ++i) { cout <<
arr[i] << " ";
} cout <<
endl;
return 0;
}
```

Output:



```
Output
Enter the number of elements: 4
Enter 4 integers: 99
5
6
35
Sorted array: 5 6 35 99
```

Time complexity:

```

→ for (i=0; i<n-1; i++) {
    for (j=0; j<n-i-1; j++) {
        if (arr[j] > arr[j+1]) {
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

```

$T(n) = (n-1)(n-1)$
 $= (n-1)^2$
 $= n^2 - n$
 $= n(n-1)$
 $= n^2$
 $T(n) = O(n^2)$

Best case	Average case	Worst case
The array is sorted n-1 comparisons $T(n) = n-1$ $T(n) = O(n)$	$(n-1) + (n-1) + (n-3) \dots + 2+1$ $1+2+\dots+(n-1)$ $T(n) = n(n-1)/2 = \frac{n^2-n}{2}$ $T(n) = O(n^2)$	The array is in reverse order Ex: 50 40 30 20 10 $(n-1) + (n-2) + \dots + 2+1$ $= n(n-1)/2 = \frac{n^2-n}{2}$ $T(n) = O(n^2)$

Conclusion:

Bubble Sort is a simple and easy-to-understand sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order. In C++, it can be implemented using loops and a temporary variable. Although it is not efficient for large datasets because its time complexity is $O(n^2)$, it is useful for learning the basic concepts of sorting, comparison, and swapping.

Selection sort

Algorithm for selection sort:

② Input:- unsorted array elements
output:- sorted array

step ①:- for $i = 0$ to $i = n - 1$ then

step ②:- select $min = i$ then do

step ③:- for $j = i + 1$ to $j = n$ then

step ④:- if $a[j] < a[min]$ then do

step ⑤:- $min = j$

step ⑥:- stop if and for

step ⑦:- if $min \neq i$ then do

step ⑧:- swap both

$temp = a[i];$

$a[i] = a[min];$

$a[min] = temp;$

step ⑨:- stop if

Code for selection sort:

```
#include <iostream> #include <vector> void
selectionSort(std::vector<int>& arr) {  int
n = arr.size();  for (int i = 0; i < n - 1;
++i) {      int minIndex = i;      for (int j =
i + 1; j < n; ++j) {          if (arr[j] <
arr[minIndex]) {              minIndex = j;
          }
      }
      if (minIndex != i) {
std::swap(arr[i], arr[minIndex]);
      }
  }
}

int main() {
int n;
    std::cout << "Enter the number of elements: ";
std::cin >> n;
    if (n <= 0)
    {
return 0;
    }

    std::vector<int> arr(n);    std::cout <<
"Enter " << n << " integers: ";
for (int i = 0; i < n; ++i) {        std::cin >> arr[i];
    }

    selectionSort(arr);

    std::cout << "Sorted array: ";
for (int i = 0; i < n; ++i) {
std::cout << arr[i] << " ";
}    std::cout << std::endl;
return 0;
}
```

Output:

```

Output
Enter the number of elements: 5
Enter 5 integers: 1
02
88
6
-8
Sorted array: -8 1 2 6 88
    
```

Time complexity:

```

for (int i=0; i<n-1; i++) {
    min = i;
    for (j=i+1; j<n; j++) {
        if (A[j] < A[min]) {
            min = j;
        }
        swap(A[i], A[min]);
    }
}
T(n) = (n-1)n
      = n2 - n
T(n) = O(n2)
    
```

<u>Best case</u>	<u>Average case</u>	<u>Worst case</u>
Perform all comparison	check every pass	All comparison
$(n-1) + (n-2) + \dots + 1$	$T(n) = (n-1) + (n-2) + \dots + 1$	$T(n) = O(n^2)$
$T(n) = n(n-1)/2$	$= n^2 - n/2$	
$= n^2 - n/2$	$T(n) = O(n)$	
$T(n) = O(n^2)$		

Conclusion:

Selection Sort is a simple comparison-based sorting algorithm. It repeatedly finds the smallest element from the unsorted part of the array and places it at the beginning.

Insertion sort

Algorithm:

③ Input:- Enter unsorted array elements
output:- sort Elements

① step ①:- for $i=1$ to len then
step ②:- $temp = a[i]$ and $j = i - 1$ then
step ③:- while $j \geq 0$ and $a[j] > temp$
step ④:- $a[j+1] = a[j]$ and $j = j - 1$
step ⑤:- stop while
step ⑥:- $a[j+1] = temp$
step ⑦:- stop for

Code:

```
#include <iostream> #include <vector> void
insertionSort(std::vector<int>& arr) { int
n = arr.size(); for (int i = 1; i < n;
++i) { int key = arr[i]; int j = i - 1;
while (j >= 0 && arr[j] > key) {
arr[j + 1] = arr[j]; j = j - 1; }
arr[j + 1] = key;
}
}

int main() {
int n;
std::cout << "Enter the number of elements: ";
std::cin >> n;
if (n <= 0)
{
return 0;
}
std::vector<int> arr(n); std::cout <<
"Enter " << n << " integers: "; for (int i
= 0; i < n; ++i) { std::cin >> arr[i];
}
```

```
insertionSort(arr);
```

```
std::cout << "Sorted array: ";
for (int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
}    std::cout << std::endl;
return 0;
}
```

Output:

```
Output
Enter the number of elements: 5
Enter 5 integers: 3
55
89
20
6
Sorted array: 3 6 20 55 89
```

Time complexity:

→ for (int i = 1; i < n - 1; i++) → n-1
key = A[i] ——— (1)
j = i - 1 ——— (1)
while (j ≥ 0 && A[j] > key)
A[j+1] = A[j]; ——— (1)
j--;
A[j+1] = key; ——— (1)
 $T(n) = (n-1) + 1 + 1 + 1 + 2 = n+4$

<u>Best case</u>	<u>Average case</u>	<u>worst case</u>
Array sorted	$1 + 2 + \dots + (n-1)$	$(n-1) + (n-2) + \dots + 1$
one comparison	$T(n) = \frac{n(n-1)}{2}$	$T(n) = \frac{n(n-1)}{2}$
$n-1$	$= \frac{n^2 - n}{2}$	$T(n) = O(n^2)$
$T(n) = n-1$	$T(n) = O(n^2)$	
$T(n) = O(n)$		

Conclusion:

Insertion Sort is a simple and efficient sorting algorithm for small or nearly sorted datasets. It sorts elements by inserting each element into its correct position in the already sorted portion of the array. It is easy to implement in C++ and requires $O(1)$ extra space.

Merge Sort:

Algorithm:

```

③ Input:- array element
   output:- sorted array

step ①:- mergeSort(arr, low, mid, high) then
step ②:- set i = low, j = mid + 1, k = low
step ③:- while i ≤ mid and j ≤ high then
step ④:- if arr[i] < arr[j] then
           temp[k] = arr[i]
           i = i + 1
         else
           temp[k] = arr[j]
           j = j + 1
         end if
step ⑤:- while j ≤ high then
           temp[k] = arr[j]
           j = j + 1
           k = k + 1
         end while
step ⑥:- while i ≤ mid then
           temp[k] = arr[i]
           i = i + 1
           k = k + 1
         end while
step ⑦:- mergeSort(arr, low, high) then do
step ⑧:- if low < high
           mid = (low + high) / 2
           mergeSort(arr, low, mid)
           mergeSort(arr, mid + 1, high)
         end if

```

Code:

```

#include <iostream> #include <vector> void
merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;    int n2 = right - mid;
    std::vector<int> L(n1);    std::vector<int> R(n2);    for (int i =
0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }
    int i = 0;    int j = 0;    int k =
left;    while (i < n1 && j <

```

```
n2) {    if (L[i] <= R[j]) {
arr[k] = L[i];
        ++i;    } else
{        arr[k]
= R[j];
        ++j;
        }
        ++k;
    }
    while (i < n1) {    arr[k]
= L[i];
        ++i;
        ++k;
    }    while (j < n2)
{        arr[k]
= R[j];
        ++j;
        ++k;
    }
}
```

```
void mergeSort(std::vector<int>& arr, int left, int right) {
if (left < right) {
    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);    mergeSort(arr,
mid + 1, right);

    merge(arr, left, mid, right);
}
}
```

```
int main() {
int n;    std::cout << "Enter the number of
elements: ";    std::cin >> n;
    if (n <= 0)
{        return
0;
    }

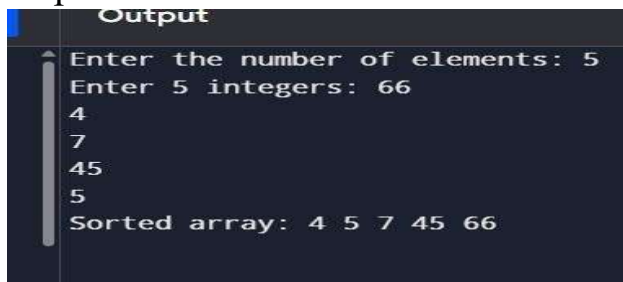
    std::vector<int> arr(n);    std::cout <<
"Enter " << n << " integers: ";    for (int i =
```

```

0; i < n; ++i) {      std::cin >> arr[i];
} mergeSort(arr, 0, n - 1);
std::cout << "Sorted array: ";  for
(int i = 0; i < n; ++i) {      std::cout
<< arr[i] << " ";    }  std::cout <<
std::endl;  return 0;
}

```

Output:



```

Output
Enter the number of elements: 5
Enter 5 integers: 66
4
7
45
5
Sorted array: 4 5 7 45 66

```

Time Complexity

$$\begin{aligned}
 & \rightarrow \text{mergeSort}(A, \text{low}, \text{high}) \{ \\
 & \quad \text{is } (\text{low} < \text{high}) \{ \\
 & \quad \quad \text{mid} = (\text{low} + \text{high}) / 2; \\
 & \quad \quad \text{mergeSort}(A, \text{low}, \text{mid}); \\
 & \quad \quad \text{mergeSort}(A, \text{mid} + 1, \text{high}); \\
 & \quad \quad \text{mergeSort}(A, \text{low}, \text{mid}, \text{high}); \\
 & \quad \} \\
 & \} \\
 & T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) + n & n>1 \end{cases} \\
 & T(n) = 2T(n/2) + n \\
 & T(n) = 2T(n/2) + n \\
 & T(n) = 2^2 [2 + (n/2) + n/4] + 2n \\
 & T(n) = 2^3 T(n/5) + 2n \\
 & \quad = 2^4 T(n/16) + 4n \\
 & \quad = 2^k T(n/2^k) + kn \\
 & \quad n=1 \\
 & \quad \frac{n}{2^k} = 1 \\
 & \quad n = 2^k \\
 & \quad \log n = \log 2^k \\
 & \quad \log n = k \log 2 \\
 & \quad k = \log_2 n \\
 & T(n) = n (T(n) + n \log_2 n) \\
 & \quad = n(1) + n \log_2 n \\
 & \quad = T(n) - O(n \log_2 n)
 \end{aligned}$$

<u>Best case</u> divides array into two halves $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \dots \rightarrow 1$ $T(n) = n \times \log_2 n$ $T(n) = O(n \log n)$	<u>Average case</u> $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots 1$ $T(n) = n \times \log_2 n$ $T(n) = O(n \log n)$
<u>worst case</u> $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots 1$ $T(n) = n \log_2 n$ $T(n) = O(n \log n)$	

Conclusion:

Merge Sort is an efficient and reliable sorting algorithm based on the divide-and-conquer technique. It divides the array into smaller parts, sorts them recursively, and then merges the sorted parts. Merge Sort has a time complexity of $O(n \log n)$ in the best, average, and worst cases. It is suitable for sorting large datasets and provides consistent performance.

Quicksort:

Algorithm:

① Input:- unsorted array elements
 output:- sorted array
 step ①:- partition (arr, low, high)
 step ②:- set pivot = arr[high]
 step ③:- for j = low to high - 1 then
 step ④:- if arr[j] < pivot then
 i = i + 1
 swap arr[i] and arr[j]
 end if
 end for
 step ⑤:- swap arr[i + 1] and arr[high]
 step ⑥:- return i + 1
 step ⑦:- quicksort(arr, low, high) then
 step ⑧:- end if

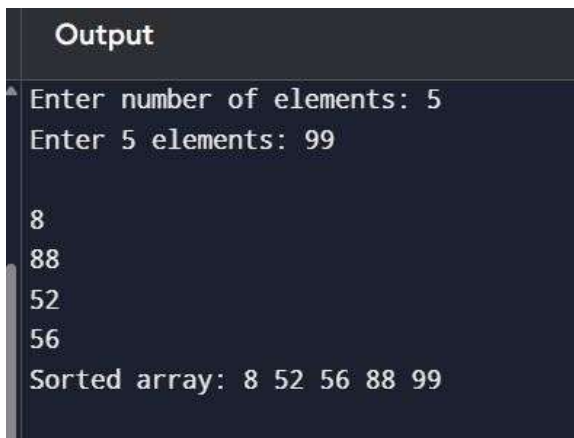
Code:

```

#include <iostream> #include <vector> void
swap(int& a, int& b) { int temp = a; a = b; b =
temp; } int partition(std::vector<int>& arr, int low, int
high) { int pivot = arr[high]; int i = low - 1;
for (int j = low; j < high; j++) { if (arr[j] < pivot)
{ i++;
swap(arr[i], arr[j]);
}
} swap(arr[i + 1], arr[high]);
return i + 1;
} void quickSort(std::vector<int>& arr, int low, int high)
{ if (low < high) { int pi = partition(arr, low,
high); quickSort(arr, low, pi - 1); quickSort(arr,
pi + 1, high);
  
```

```
    }  
} int main()  
{    int n;  
std::cout <<  
"Enter  
number of  
elements:  
";  
std::cin >> n;  
  
    std::vector<int> arr(n);    std::cout <<  
"Enter " << n << " elements: ";    for (int i =  
0; i < n; i++) {        std::cin >> arr[i];  
}    quickSort(arr, 0, n - 1);  
std::cout << "Sorted array: ";    for  
(int i = 0; i < n; i++) {        std::cout  
<< arr[i] << " ";    }    std::cout <<  
std::endl;    return 0;  
}
```

Output:



```
Output  
Enter number of elements: 5  
Enter 5 elements: 99  
  
8  
88  
52  
56  
Sorted array: 8 52 56 88 99
```

Time complexity :

void quicksort(int arr[], int low, int high) { $\rightarrow T(n)$

if (low < high) $\rightarrow (1)$

partition (A, low, high) $\rightarrow n$

quicksort (A, low, p-1) $\rightarrow T(n/2)$

quicksort (A, p+1, high) : $\rightarrow T(n/2)$

$$T(n) = T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n$$

$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) & n>1 \end{cases}$$

$$T(n) = 2T(n/2) + n \quad D > -1$$

$$a=2 \quad b=2 \quad k=1 \quad p=0$$

$$\log_a b = \log_2 2 = 1 = k$$

$$T(n) = O(n^k \log^{p+1} n)$$

$$T(n) = O(n \log n)$$

Best case

pivot divides array

two equal parts

$$n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 + n/4 + n/4 \dots$$

$$T(n) = n \times \log_2 n$$

$$T(n) = n \log n$$

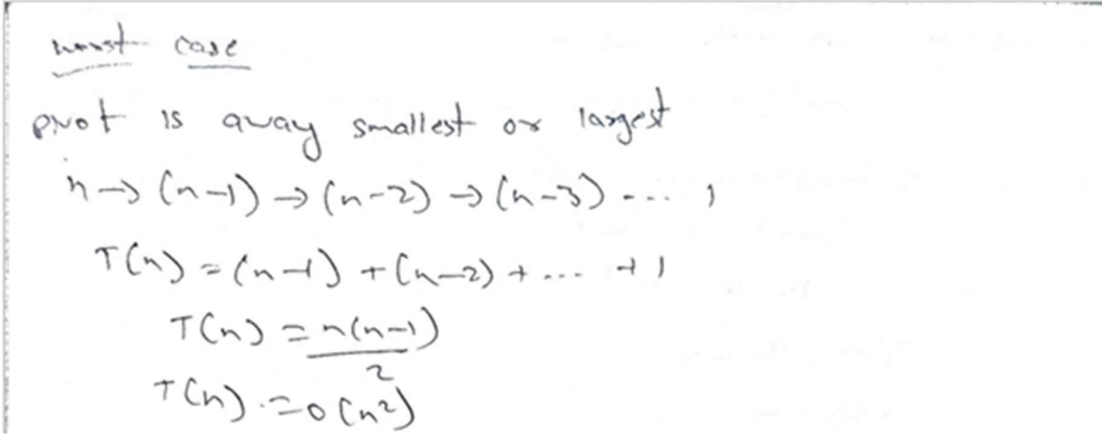
Average case

divides into balance parts

$$n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 + n/4 + n/4 \dots$$

$$T(n) = n \times \log_2 n$$

$$T(n) = O(n \log n)$$



Handwritten text on a whiteboard background:

Worst case
pivot is away smallest or largest
 $n \rightarrow (n-1) \rightarrow (n-2) \rightarrow (n-3) \dots 1$
 $T(n) = (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

Conclusion:

Quick Sort is an efficient sorting algorithm based on the divide-and-conquer technique. It selects a pivot element and partitions the array around the pivot, then recursively sorts the smaller sub-arrays. Its average time complexity is $O(n \log n)$, while the worst-case complexity is $O(n^2)$. Quick Sort is widely used because it is fast, memory-efficient, and suitable for sorting large datasets.